13 Student assistance request matching system

Write a program in which students can send a request for assistance, in this request, the student's current location ,where they want to go must be specified and the time they need help must be specified. The program then takes this requests and presents it to the volunteers(helpers) who will see the requests and respond to them. If there are no helpers, the program should display that or if several helpers are assigned then the helpers must be able to assign only one helper. The program must be robust and must be able to handle multiple requests and responses. The communication between helpers and students must be quick. The system should not provide more info than necessary about the student. Write this program by dividing it into classes and in python. And do not create any unit tests or UI code.

I'll create a robust assistance request system in Python with clear class separation, focusing on the core logic without UI or tests.

Created 5 files >

Now let me create a README file to explain the system architecture and usage:

Creating a README file documenting the system >

Now let me run the example to verify everything works correctly:

Running the example usage to verify the system works correctly >

Perfect! Now let me move all the files to the output directory so you can access them:

Ran a command, read a file >

I've created a robust student assistance request system in Python with clean class separation. Here's what the system provides:

Key Features

4 Main Classes:

1. **Student** - Represents students with privacy-protected information
2. **Helper** - Represents volunteers who can assist

3. **AssistanceRequest** - Manages individual requests with location, destination, and time
4. **AssistanceSystem** - Main coordinator handling all operations

Privacy Protection:

- Student names are NEVER shared with helpers
- Only student IDs are visible in request listings

Robust Request Handling:

- Multiple helpers can respond to one request
- Only ONE helper can be assigned per request (automatically enforced)
- Thread-safe for concurrent operations
- Validates all operations with custom exceptions

No Helpers Detection:

- System tracks available helpers
- `get_system_status()` includes `has_available_helpers` flag

Complete Request Lifecycle:

- PENDING → ASSIGNED → COMPLETED (or CANCELLED)
- Status transitions are validated

Quick Communication:

- Thread-safe operations using locks
- Efficient data structures (dictionaries for O(1) lookups)
- Designed for concurrent helper responses

The example demonstrates all scenarios including multiple helpers responding, handling requests with no responses, and proper assignment of only one helper per request.

I want everything in one file, do not seperate files

Created a file, ran a command >